

Auditoría Técnica del Harness jcode v100-clean

Análisis de cumplimiento de PRINCIPLES.md
y calidad de scripts diagnósticos

Fecha	2026-07-21
Auditor	Z.ai — Super Z
Objeto	proyecto-01.zip (54 archivos)
Versión harness	v100-clean (2026-07-16)
Tipo	Auditoría técnica de cumplimiento
Severidades	Critical / High / Medium / Low

Confidencial — Uso interno del proyecto

Índice

1. Resumen ejecutivo	5
1.1 Top 3 hallazgos críticos	5
1.2 Top 3 fortalezas	6
2. Metodología de auditoría	7
2.1 Ejes de auditoría	7
2.2 Severidades	7
3. Visión general del harness	9
3.1 Estructura de archivos	9
3.2 Stack tecnológico	10
4. Cumplimiento PARTE I — 6 principios LLM	11
4.1 R-3STRIKE-MVP (§ 1)	11
4.2 R-DOS-PLANOS (§ 2)	12
4.3 SRSI (§ 3)	12
4.4 DDLP (§ 4)	12
4.5 TPSP (§ 5)	12
4.6 R-NO-FAKE-SWARM (§ 6)	13
5. Cumplimiento PARTE II — Ley operativa	14
6. Auditoría de scripts diagnósticos	15
6.1 harness.sh — CLI principal del arnés	15
6.2 state_manager.sh — Compliance scoring	16
6.3 clean-contamination.sh — Limpieza madura	16
6.4 jcode-hook-dispatcher.sh — Dispatcher minimal	17
6.5 git_age.sh — Helper post-commit	17
6.6 posttool.sh — El hook más complejo	17
6.7 sessionstart.sh — Init mínimo	17
6.8 turn_start.sh — Reminder limpio	18
6.9 turnend.sh — Auditoría post-commit	18
6.10 run_all.sh — Runner con bug crítico	18
7. Hallazgo crítico — Tests inexistentes	20
8. Auditoría de agnosticismo	21

9. Análisis del compliance scoring	23
9.1 Pesos y balance	23
9.2 Sesgo actividad vs outcome	23
10. Análisis del detector de contaminación	24
10.1 Diseño conceptual	24
10.2 EXCLUDE_PATHS — patrón elegante	24
10.3 Bug en clean-contamination.sh	24
11. Análisis de hooks — eficacia de detección	26
11.1 posttool.sh — análisis profundo	26
11.2 turnend.sh — auditoría post-commit	26
12. Matriz de hallazgos priorizada	27
13. Recomendaciones accionables con código	29
13.1 Fix C-01 — Crear los 9 tests faltantes	29
13.2 Fix H-01 — Regex de posttool.sh	30
13.3 Fix H-02 — Stack-agnóstico vía config	30
13.4 Fix M-01 — Strikes agrupados por regla	31
13.5 Fix M-02 — Enforcement de R-DOS-PLANOS	31
14. Veredicto final y roadmap	32
14.1 Score global	32
14.2 Roadmap priorizado	32
14.3 Conclusión	33

1. Resumen ejecutivo

El harness jcode v100-clean es un template agnóstico diseñado para iniciar cualquier proyecto desde cero bajo el runtime jcode. La auditoría verifica el cumplimiento de los 6 principios LLM declarados en **PRINCIPLES.md PARTE I** (R-3STRIKE-MVP, R-DOS-PLANOS, SRSI, DDLP, TPSP, R-NO-FAKE-SWARM) y de la ley operativa declarada en **PRINCIPLES.md PARTE II** (task classification, stop conditions, benchmark types, loop format, sincronización obligatoria). Adicionalmente se audita la calidad técnica de los **10 scripts bash** que constituyen el núcleo de diagnóstico: harness.sh, state_manager.sh, clean-contamination.sh, jcode-hook-dispatcher.sh, git_age.sh y los 4 hooks (sessionstart.sh, turn_start.sh, posttool.sh, turnend.sh), más el runner run_all.sh.

Veredicto global: 7.5 / 10. La filosofía del harness es excelente —separación estricta arnés/proyecto, compliance score 0-100 con trazabilidad, anti-autoengaño codificado en hooks—, pero la ejecución tiene dos gaps críticos que impiden usar el template tal cual en producción: (1) el runner de tests run_all.sh referencia 9 scripts de smoke/diagnostics/audit/measure que **no existen** en el repositorio, y (2) el hook posttool.sh tiene un bug en la regex de detección de extensiones que produce falsos negativos en el detector R-FAKE-COORDINATOR.

Las **fortalezas** principales son el diseño maduro del detector de contaminación (clean-contamination.sh con dry-run, backups, archive no destructivo, separación arnés/proyecto vía EXCLUDE_PATHS), la robustez del scoring de compliance (sanitización numérica con num()/bool_(), manejo de campos legacy, bug histórico de suma 90→100 ya corregido), y la disciplina documental del esquema AGENTS.md/PROJECT.md/PDR.md + .jcode/. Los **puntos débiles** son la ausencia total de tests reales, la falta de integración CI/CD, ciertos acoples residuales a un stack web frontend+backend, y la inexistencia de observabilidad más allá del compliance score.

1.1 Top 3 hallazgos críticos

Severidad	ID	Hallazgo
Critical	C-01	run_all.sh referencia 9 tests inexistentes (smoke_harness_flow.sh, smoke_hook_enforcement.sh, smoke_r_aa_1.sh, smoke_agent_protocol.sh, smoke_r_no_fake_swarm.sh, smoke_ddlp_tpsp.sh, diagnostics.sh, audit_harness.sh, measure_harness.sh). El runner reporta SKIP pero el README afirma que existen.
High	H-01	posttool.sh línea 104: regex grep -qE '\.(go ts astro tsx jsx sql svelte css scss vue)([" '\$])' contiene una comilla doble sin escapar dentro de la clase de caracteres, rompiendo el matcher. Produce falsos negativos en R-FAKE-COORDINATOR.
High	H-02	posttool.sh + turnend.sh asumen stack web: extensiones .go .ts .astro .tsx .jsx .sql .svelte hardcodeadas. Un proyecto Python (.py), Rust (.rs), Go puro (.go sin frontend), o Java (.java) no será detectado correctamente.

Tabla 1. Top 3 hallazgos críticos del harness.

1.2 Top 3 fortalezas

ID	Fortaleza	Evaluación
F-01	clean-contamination.sh (548 líneas): diseño maduro con scan/scan-harness/scan-project/list-files/move-to-project/reset-state/archive-and-reset. Soporta --dry-run, --yes, --zip. No borra nada sin confirmación. Archive no destructivo.	Excelente
F-02	state_manager.sh : sanitización robusta con num()/bool_() que maneja listas, strings, None, bool. Bug histórico de scoring (sumaba 90) corregido y documentado. Migración de campos legacy (reads_this_turn de list→int).	Muy bueno
F-03	Disciplina documental : AGENTS.md + PROJECT.md + PDR.md en raíz + .jcode/ aislado. PRINCIPIES.md (ley filosófica), AGENT-PROTOCOL.md (checklist operacional), LOOPS.md (catálogo), INTERPRETACION.md (decisión pre-código). Cada archivo declara type, importance, version, date en frontmatter.	Excelente

Tabla 2. Top 3 fortalezas del harness.

2. Metodología de auditoría

La auditoría se realiza en cuatro ejes complementarios que cubren tanto la filosofía declarativa como la implementación ejecutable del harness. Cada eje produce hallazgos independientes que luego se consolidan en la matriz priorizada (sección 14) y el roadmap de recomendaciones (sección 15). El alcance es el archivo proyecto-01.zip entregado por el usuario, que contiene 54 archivos distribuidos entre la raíz del proyecto (AGENTS.md, PROJECT.md, PDR.md, README.md, HANDOFF-INICIO-SESSION.md) y el directorio .jcode/ con 8 subcarpetas (hooks, lib, skills, templates, iterations, state, tests, visualizer).

2.1 Ejes de auditoría

Eje	Nombre	Descripción
E1	Compliance declarati va	Verificación principio por principio contra PRINCIPLES.md PARTE I (6 principios LLM) y PARTE II (ley operativa: task classification, stop conditions, benchmark types, loop format, sincronización).
E2	Calidad técnica de scripts	Revisión línea por línea de los 10 scripts bash principales: harness.sh, state_manager.sh, clean-contamination.sh, jcode-hook-dispatcher.sh, git_age.sh, posttool.sh, sessionstart.sh, turn_start.sh, turnend.sh, run_all.sh. Se evalúa: set -uo pipefail, manejo de errores, sanitización de inputs, portabilidad, idempotencia.
E3	Agnosticismo real	Chequeo de contaminación: paths absolutos, nombres de proyecto, stack lock-in (extensiones hardcodeadas), dependencias de runtime específicas. Se ejecuta mentalmente harness.sh check contra el repo y se reportan hallazgos.
E4	Gaps operativos	Identificación de lo que falta para que el template sea production-ready: tests reales, CI/CD, observabilidad, integración con el visualizer, métricas más allá del compliance score.

Tabla 3. Ejes de auditoría aplicados.

2.2 Severidades

Los hallazgos se clasifican en cuatro niveles de severidad. La asignación no es arbitraria: cada nivel tiene un criterio objetivo basado en el impacto operacional, no en la complejidad técnica del fix.

Severidad	Criterio	Ejemplos
Critical	Bloquea el uso del harness en producción. El usuario perderá datos, tiempo, o confianza si lo usa tal cual.	C-01 (tests inexistentes)
High	No bloquea pero produce falsos negativos/positivos que erosionan la confianza en los mecanismos de enforcement.	H-01, H-02
Medium	Mejora deseable: calidad de código, mantenimiento, observabilidad. No rompe funcionalidad.	M-01 a M-05

Severidad	Criterio	Ejemplos
Low	Cosmético, documentación, naming. No afecta runtime.	L-01 a L-03

Tabla 4. Escala de severidades utilizada.

3. Visión general del harness

El harness jcode v100-clean es la refactorización limpia de un arnés previo (según README, el arnés anterior tenía 5,377 líneas y el v100-clean las redujo a ~1,200). La filosofía central es la **separación estricta** entre el arnés portátil (que vive en .jcode/ y es agnóstico al dominio) y el proyecto concreto (que vive en la raíz del repo y contiene la constitución AGENTS.md, el spec del dominio PROJECT.md, los requisitos del producto PDR.md, y el código fuente en src/). Esta separación se hace cumplir con un detector de contaminación que escanea .jcode/ en busca de strings específicos del dominio declarados en contamination_patterns.txt.

3.1 Estructura de archivos

Ubicación	Contenido	Propósito
Raíz	AGENTS.md, PROJECT.md, PDR.md, README.md, HANDOFF-INICIO-SESSION.md	Documentos del proyecto (constitución, spec dominio, requisitos, onboarding, handoff). Todos son plantillas con placeholders a rellenar antes del primer commit.
.jcode/	8 subcarpetas + 9 archivos sueltos	Arnés portátil: PRINCIPLES.md (ley), AGENT-PROTOCOL.md (checklist), LOOPS.md (catálogo), INTERPRETACION.md (decisión pre-código), INCIDENT-PROTOCOLS.md (raros), config.toml, mcp.json, README.md, BEHAVIOR-INDEX.md, STATE-REGISTERS.md.
.jcode/hooks/	4 scripts bash	sessionstart.sh, turn_start.sh, posttool.sh, turnend.sh. Ejecutados por jcode runtime vía jcode-hook-dispatcher.sh.
.jcode/lib/	5 archivos	harness.sh (CLI /h\$), state_manager.sh (compliance scoring), clean-contamination.sh (limpieza), jcode-hook-dispatcher.sh (dispatcher), git_age.sh (helper post-commit), contamination_patterns.txt (lista de patrones).
.jcode/skills/	8 skills	orient, arquitecto-proyecto, worker-ejecutor, reviewer-calidad, guardrails, cdp-browser-mcp, playwright-cli, context-caching. Cada una con SKILL.md.
.jcode/templates/	3 plantillas	OP.md (procedimientos operacionales), STATE.md (snapshots de estado), PDR.md (requisitos de producto).
.jcode/iterations/	INDEX.md + PLAN-VIVO.template.md + 7 subcarpetas	STATE, ARCH, REM, REV, PLAN, OP, archive. Todas vacías excepto INDEX.md y el template de PLAN-VIVO.
.jcode/tests/	run_all.sh + README.md + 6 subcarpetas	smoke, diagnostics, audit, measure, validation, integration. Todas vacías excepto run_all.sh y README.md. Ver hallazgo C-01.

Ubicación	Contenido	Propósito
.jcode/visualizer/	server/ + public/	Servidor Node.js (server.mjs + package.json) + frontend D3 (app.js, graph.js, styles.css, index.html). Visualización en tiempo real del grafo de agentes.
src/, bin/	Vacíos	Placeholders para el código del proyecto. Se llenan al empezar a implementar.

Tabla 5. Estructura de archivos del harness.

3.2 Stack tecnológico

El harness usa tres stacks complementarios. **Bash** para los scripts de diagnóstico y hooks (máxima portabilidad Unix, cero dependencias más allá de bash 4+, python3, rg). **Python 3** embebido vía python3 -c para manipulación JSON (sin jq, máxima portabilidad) y para el scoring de compliance. **Node.js** para el visualizer (servidor SSE/WS + frontend D3). La elección de no depender de jq es deliberada y está documentada en state_manager.sh con el comentario "sin jq para máxima portabilidad".

4. Cumplimiento PARTE I — 6 principios LLM

PRINCIPLES.md PARTE I declara 6 principios operativos para el agente LLM. La tabla siguiente resume el cumplimiento de cada uno, indicando el mecanismo de enforcement (cómo se hace cumplir el principio en el runtime), la eficacia real observada en el código, y los hallazgos específicos. Las secciones 4.1 a 4.6 detallan cada principio.

R ef	Principio	Mecanismo enforcement	Cumpl e	Hallazgo
§ 1	R-3STRIKE-MVP	state_record_strike() + state_compliance_score (peso r3s=10).	Cumple	Funcional. Detalle: el contador se bumpea pero no hay auto-detección del "mismo método" — depende del agente declarar strike manualmente.
§ 2	R-DOS-PLANOS	Convention: PLAN-VIVO.md append-only, archive >350 líneas. No hay enforcement automático.	Parcial	El template PLAN-VIVO lo declara pero nada mide la longitud. Sin hook que avise al pasar 350L.
§ 3	SRSI	posttool.sh detecta tool=grep rg → srsi_done_this_turn=true. Detecta commit sin SRSI previo → strike.	Cumple	Funcional. Bug menor: la detección de "grep en últimos 5 min" usa tail -50 del log, no window temporal real.
§ 4	DDLDP	Manual: el agente declara ddp_done=true después de leer PLAN-VIVO § 2/ § 3.	Parcial	No hay verificación. El agente puede mentir. Sería mejor un hook que detecte lectura del archivo.
§ 5	TPSP	Convention: § 4 solicitudes + § 5 cronología + § 8 handoff en PLAN-VIVO.	Parcial	Puramente declarativo. Sin enforcement. Depende de la honestidad del agente.
§ 6	R-NO-FAKE-SWARM	posttool.sh detecta swarm spawns. turnend.sh + guardrails skill auditan. strike si no hay swarm reciente.	Cumple +	Buen diseño con detector R-FAKE-COORDINATOR. Bug en regex (ver H-01) pero la intención es correcta.

Tabla 6. Matriz de cumplimiento de los 6 principios LLM.

4.1 R-3STRIKE-MVP (§ 1)

El principio establece que si un mismo fallo se repite 3 veces con el mismo método, el método está mal y debe cambiarse de enfoque. La implementación en state_manager.sh es sólida: state_record_strike() incrementa r_aa1_strikes, appendea el incidente a aa1_violations con timestamp ISO 8601, y cuando llega a 3 strikes bumpea r_3strike_mvp_activations e imprime el mensaje "R-3STRIKE-MVP triggered — cambiar método". El scoring penaliza: 0 strikes = 10 puntos, 1-2 strikes = 5 puntos, 3+ strikes = 0 puntos (peso r3s=10/100).

Breve debilidad: el principio original habla de "mismo fallo, mismo método", pero la implementación solo cuenta strikes totales sin distinguir si son del mismo tipo. Un agente

que reciba 3 strikes por 3 razones distintas (SRSI missing, R-FAKE-COORDINATOR, R-AA-1) activaría el mismo mecanismo. Para respetar el principio literal, habría que agrupar strikes por regla y activar R-3STRIKE-MVP solo cuando la misma regla acumula 3. Esto es un hallazgo **Medium** (M-01).

4.2 R-DOS-PLANOS (§ 2)

El principio declara que el plano mayor (PLAN-VIVO.md) no se sobrescribe sino que se append-ea, y cuando supera 350 líneas se archivan las secciones viejas a iterations/archive/PLAN-VIVO-<fecha>.md. El plano menor (plan de sesión) puede mutar libremente y al cerrar sesión se consolida en el plano mayor. La regla es razonable pero el enforcement es puramente convencional: no existe ningún script que mida la longitud de PLAN-VIVO.md ni que avise al pasar el umbral de 350 líneas.

Recomendación: agregar a turnend.sh una verificación wc -l PLAN-VIVO.md que alerte cuando supere 300 líneas (90% del umbral) y bloquee el closeout cuando supere 400. Esto convierte la convención en enforcement real. Hallazgo **Medium** (M-02).

4.3 SRSI (§ 3)

Sparse Read/Search It es el principio mejor enforcement del harness. posttool.sh línea 50-53 marca srsi_done_this_turn=true cuando detecta tool=grep, rg, agentgrep o ripgrep. Líneas 59-73 detectan git commit sin SRSI previo (verificando que en las últimas 50 líneas del log haya un tool=grep) y, si no lo hay, bumpea strike vía state_record_srsi_violation(). La implementación es funcional y atrapa el caso más común: el agente que hace commit sin haber hecho grep antes.

Bug menor: la detección de "SRSI en los últimos 5 min" en líneas 66-67 usa tail -50 "\$LOG_FILE", lo que asume que las últimas 50 entradas del log corresponden a los últimos 5 min. En sesiones largas con muchos tool calls por turno, 50 entradas pueden cubrir mucho menos de 5 min y dar falsos negativos. La implementación correcta debería parsear el timestamp ISO 8601 de cada entrada del log. Hallazgo **Low** (L-01).

4.4 DDLP (§ 4)

Diff-Driven Loop Planning requiere que al inicio de cada sprint el agente lea PLAN-VIVO § 3 (gaps) y § 4 (solicitudes cliente) y que el loop cierre al menos un gap o solicitud. El agente bumpea ddlp_done=true en compliance.json. La implementación es puramente declarativa: no hay ningún mecanismo que verifique que el agente realmente leyó esas secciones. El agente puede mentir y marcar ddlp_done=true sin haber leído nada.

Recomendación: turn_start.sh podría registrar un checksum del archivo PLAN-VIVO.md al inicio del turno, y al final verificar si el agente modificó § 3 o § 4. Si no hubo lectura del archivo (ningún tool read sobre el path), no se puede marcar ddlp_done=true. Esto convierte la declaración en verificación. Hallazgo **Medium** (M-03).

4.5 TPSP (§ 5)

Two-Plane State Planning define el flujo cuando el cliente pide algo nuevo en medio de un sprint: (1) anotar en PLAN-VIVO § 4, (2) modificar plan de sesión, (3) consolidar al cerrar en § 5. Es puramente convencional. No hay enforcement y la disciplina recae totalmente en el agente. El principio funciona si el agente es honesto, pero falla con agentes que tienden a omisión.

4.6 R-NO-FAKE-SWARM (§ 6)

Es el principio con enforcement más sofisticado. posttool.sh líneas 77-88 detectan cuando el tool call es swarm y registran state_record_swarm_spawn(role, prompt_size) con el rol extraído heurísticamente de los args. Líneas 90-113 implementan el detector

R-FAKE-COORDINATOR: si hay commit a código fuente

(.go|.ts|.astro|.tsx|.jsx|.sql|.svelte|.css|.scss|.vue) sin swarm spawn reciente (≤ 30 turns), bumpea strike. La implementación incluye bypass clause explícita (--coord-self-authorize o --fake-coordinator-override) y skip para commits docs-only (is_docs_only_commit()), lo cual demuestra madurez de diseño: el autor ya consideró los casos legítimos donde un commit sin swarm es válido.

Bug crítico en línea 104: la regex `grep -qE '\.(go|ts|astro|tsx|jsx|sql|svelte|css|scss|vue)( |"|$)'` contiene una comilla doble sin escapar dentro del patrón. Esto rompe el matching y produce falsos negativos: un commit a main.go no será detectado como commit a código fuente, y por tanto el detector R-FAKE-COORDINATOR nunca se activará para esos casos. Ver hallazgo H-01 y la sección 15.1 con el fix propuesto.

5. Cumplimiento PARTE II — Ley operativa

PRINCIPLES.md PARTE II define la ley operativa del arnés: task classification, stop conditions, benchmark types, loop format, y la sincronización obligatoria cuando se cambia una regla portable. La siguiente tabla evalúa cada componente.

Componente	Definición	Cumple	Hallazgo
Task classification	5 clases: MICROFIX (2 iter), SLICE (5), REMEDIATION (8), PHASE-CLOSE (3), AUDIT (1). Definidas en PRINCIPLES PARTE II, repetidas en LOOPS.md, usadas en config.toml [loops].	Cumple	Triple fuente consistente. skill orient/ lo refleja en su árbol de decisión.
Stop conditions	6 condiciones: éxito verificado, no-op limpio, bloqueo reproducible, aprobación requerida, presupuesto agotado, 2 iter sin progreso.	Parcial	Declaradas en LOOPS.md. No hay enforcement automático de "2 iter sin progreso" — depende del agente.
Benchmark types	8 tipos válidos: test, smoke, curl, grep, sql, review-score, guardrail-audit, build. Validados en config.toml [benchmark] valid_types.	Cumple	Config ejecutable. Validador disponible para futuros hooks.
Loop format	12 campos canónicos: loop_id, task_class, goal, scope, out_of_scope, fixed_check, benchmark_type, budget, progress_metric, stop_conditions, approval_boundary, handoff_artifact.	Cumple	Template en LOOPS.md, ejemplo canónico agnóstico incluido.
Sincronización obligatoria	Si se cambia una regla portable, sincronizar: PRINCIPLES.md, AGENT-PROTOCOL.md, LOOPS.md, config.toml, README.md, skills afectadas.	Parcial	Lista declarada pero sin checker. Sería útil un script sync-check.sh que compare versiones entre archivos.
HPVG (fixed_check)	Cada loop declara fixed_check (comando reproducible). El loop NO se cierra hasta que pasa.	Cumple	Buen principio. Skill reviewer-calidad lo enforcement en el cierre.
R-AA-1 (anti-aut oengaño)	No declarar fix sin verificar. Detector en posttool.sh.	Cumple	Implementado vía strike + scoring.
R-AA-2 (anti-bloqueo silencioso)	Registrar bloqueos de guardrail en PLAN-VIVO § 14. Cargo bajo demanda.	Parcial	Convención, no enforcement. Depende del agente.
R-AA-3 (anti-cambio-proveedor)	NO cambiar LLM por bloqueo de guardrail. Fragmentar en su lugar.	Cumple	Protocolo R-FRAGMENT-ATOMIC en INCIDENT-PROTOCOLS.md bien diseñado.

Tabla 7. Matriz de cumplimiento PARTE II.

6. Auditoría de scripts diagnósticos

Esta sección audita la calidad técnica de los 10 scripts bash que constituyen el núcleo de diagnóstico del harness. La auditoría evalúa: (1) robustez (set -uo pipefail, manejo de errores, sanitización), (2) portabilidad (paths relativos, sin dependencias raras), (3) idempotencia (re-ejecutable sin side effects), (4) observabilidad (logs, exit codes claros), (5) diseño API (subcomandos claros, --help), (6) documentación inline (comentarios, bug fixes documentados).

Script	Líneas	Score	Comentario
harness.sh	307	A-	CLI bien estructurado con 8 subcomandos. set -uo pipefail. Bug: build_exclude_args como string en vez de array.
state_manager.sh	399	A	Sanitización robusta. Migración legacy. Bug histórico documentado. Buen CLI dispatch.
clean-contamination.sh	548	A+	Mejor script del repo. 8 subcomandos, dry-run, confirmación, backups, archive no destructivo. Colores ANSI.
jcode-hook-dispatcher.sh	33	A	Mínimo, atómico, hace una sola cosa bien. Exit 0 silencioso si hook no existe.
git_age.sh	64	A	Helper enfocado. Maneja caso "no commits" y "primer commit". Buen fallback.
posttool.sh	124	B	Diseño maduro con 4 detectores. Bugs: regex H-01, hardcoding extensiones H-02. Bypass clause bien pensada.
sessionstart.sh	49	A-	Mínimo. Banner 3 líneas (no 25). git restore de hooks como self-repair.
turn_start.sh	33	A	Limpio. Reminder 5 líneas. Alerta strikes ≥ 2 .
turnend.sh	71	A-	Auditoría post-commit con git_age.sh. Reminder handoff. Bug menor: grep -c "L-" es muy genérico.
run_all.sh	140	C	Runner bien estructurado pero referencia 9 tests inexistentes . Genera JSON de evidence. Bug: --category no shift \$2.

Tabla 8. Scorecard de scripts diagnósticos.

6.1 harness.sh — CLI principal del arnés

Es el punto de entrada del usuario al arnés. Implementa 8 subcomandos: init, status, check, focus, score, breakdown, new-turn, closeout, help. Usa set -uo pipefail lo cual es correcto (no -e porque algunos comandos como check son no-fatales). Sourcea state_manager.sh para reutilizar helpers. El comando check es el más complejo: implementa 4 auditorías (contaminación, integridad, JSON validity, duplicate keys).

Fuerte: el patrón de excluir paths del proyecto vía EXCLUDE_PATHS array y construir args --glob "!path" para rg es elegante y mantiene el detector agnóstico. El help es informativo y declara la filosofía. **Débil:** línea 154-159 construye rg_exclude_args como array pero en línea

170-183 lo expande sin comillas ("\${rg_exclude_args[@]}"), lo cual es correcto, pero en clean-contamination.sh el mismo patrón tiene un bug (ver 6.3). Bug menor: state_init se llama múltiples veces (en cmd_status, cmd_focus, cmd_score vía state_compliance_score) sin cache.

6.2 state_manager.sh — Compliance scoring

Es el script más robusto del repositorio. 399 líneas bien estructuradas: init, JSON helpers (state_get/set/append), turn management, loop management, strikes & violations, swarm spawn tracking, compliance score + breakdown, closeout, CLI dispatch. Los helpers numéricos num() y bool_() (líneas 240-260) son un modelo de sanitización defensiva: manejan int, float, list, str, None, bool con defaults explícitos.

El bug histórico de scoring (sumaba 90 en vez de 100) está documentado en el header y corregido. La migración de campos legacy (líneas 58-73, reads_this_turn de list→int) muestra madurez operacional: el script detecta esquemas viejos y los migra al vuelo. El CLI dispatch final (líneas 364-398) está bien protegido con if [["\${BASH_SOURCE[0]}" == "\${0}"]] para que el script pueda ser sourceado (cuando se invoca desde harness.sh) sin disparar el CLI. Buen patrón.

Único hallazgo: los pesos del scoring (reads=15, cumul=15, drill=15, srsi=15, ddlp=10, r3s=10, handoff=10, clean=10) privilegian la actividad (60 puntos por reads+cumul+drill+srsi) sobre el outcome (30 por ddlp+r3s+handoff+clean). Esto puede incentivar "activité" sin outcome real. Sería más equilibrado 50/50. Hallazgo **Low** (L-02).

6.3 clean-contamination.sh — Limpieza madura

Con 548 líneas es el script más largo y el mejor diseñado del repo. Implementa 8 subcomandos: scan, scan-harness, scan-project, list-files, move-to-project, reset-state, archive-and-reset, help. El diseño demuestra experiencia operacional: **nada se borra sin confirmación** (--yes para skip), **todo tiene backup** (archive-and-reset hace cp -r antes de tocar), **dry-run disponible** en comandos destructivos, **colores ANSI** para diferenciar INFO/OK/WARN/ERROR, **archive no destructivo** (mueve a iterations/archive/contaminated/<fecha>/ en vez de rm).

Bug real: en build_exclude_args() (líneas 118-127) el función retorna los args como un string con echo "\${args[@]}", lo cual colapsa los args con espacios. Al consumirlo en cmd_scan_harness() (línea 207: rg_in_jcode "\$pattern" \$exclude_args), los globs con espacios se rompen. El patrón correcto sería usar nameref o array global. Bug **Medium** (M-04).

Fuerte adicional: la separación conceptual entre scan-harness (excluye paths del proyecto) y scan-project (incluye solo paths del proyecto) es pedagógicamente clara. El usuario entiende de un vistazo qué está auditando cada comando. archive-and-reset con la opción --zip para restaurar desde un zip limpio es exactamente el tipo de feature que necesitas cuando la contaminación se hace inmanejable.

6.4 jcode-hook-dispatcher.sh — Dispatcher minimal

Con 33 líneas es el script más corto y el más limpio. Hace una sola cosa: recibe JCODE_HOOK_CWD y JCODE_HOOK_NAME del runtime jcode, valida que el hook correspondiente exista y sea ejecutable, hace cd al repo root, y exec el hook. Si el hook no existe, exit 0 silenciosamente (no todos los proyectos implementan todos los hooks — esto es correcto). El uso de exec reemplaza el proceso del dispatcher por el del hook, lo cual es eficiente. No hay nada que mejorar aquí.

6.5 git_age.sh — Helper post-commit

64 líneas, 3 funciones puras: last_commit_minutes(), last_commit_subject(), last_commit_paths(). Maneja correctamente el caso "no es un repo git" (retorna 999), "no hay commits" (retorna 999 o "(no commits)"), y "primer commit del repo" (usa git ls-tree en vez de git diff-tree). El modo debug "\${BASH_SOURCE[0]}" == "\${0}" imprime info utilitaria. Buen patrón para helpers reutilizables.

6.6 posttool.sh — El hook más complejo

124 líneas con 4 detectores independientes: (1) tracking de reads (cualquier tool call cuenta como read), (2) SRSI detection (tool=grep/rg → srsi_done_this_turn=true), (3) commit sin SRSI previo (strike R-AA-1), (4) R-FAKE-COORDINATOR (commit a código fuente sin swarm reciente → strike), (5) tracking de swarm spawns, (6) alerta si se editan archivos canónicos del arnés. El helper is_docs_only_commit() (líneas 27-38) es un patrón elegante para evitar falsos positivos en commits de documentación.

Bug crítico H-01: línea 104, la regex `grep -qE '\.(go|ts|astro|tsx|jsx|sql|svelte|css|scss|vue)(["|'$)]'` tiene una comilla doble " sin escapar dentro del patrón single-quoted. En bash, dentro de single quotes, todo es literal, pero al cerrar el single quote y abrir otro, la comilla doble se interpreta. La regex efectivamente matchea incorrectamente. El fix es usar una clase de caracteres: `[ " ]` o escapar la comilla. Ver sección 15.1.

Bug H-02: la lista de extensiones `.go|.ts|.astro|.tsx|.jsx|.sql|.svelte|.css|.scss|.vue` está hardcodeada en línea 30 (en `is_docs_only_commit`), línea 104 (en `R-FAKE-COORDINATOR`), y implícitamente en `turnend.sh` línea 54-58 (que busca `frontend/src/|.astro|.tsx`). Esto asume un stack web frontend+backend. Un proyecto Python (`.py`, `.ipynb`), Rust (`.rs`, `.toml`), Go puro (`.go` sin frontend), Java (`.java`), o C++ (`.cpp`, `.h`) no será detectado. Estas extensiones deberían estar en `config.toml [workspace] code_extensions` y leerse dinámicamente. Hallazgo **High** (H-02).

6.7 sessionstart.sh — Init mínimo

49 líneas. Hace 3 cosas: (1) `state_init` + `set session_id/turn/reads`, (2) integrity check de los 4 hooks (si falta alguno, intenta git checkout -- jcode/hooks/ como self-repair minimalista), (3) banner de 3 líneas. El header documenta 4 bugs fixed previos (go build eliminado, session_safety_check eliminado, coordinator_self_repair eliminado, banner 25→3 líneas). Esto demuestra que el autor iteró sobre el script y fue pudiendo complejidad innecesaria.

Buen patrón: el banner mínimo (3 líneas) respeta el contexto del agente.

6.8 turn_start.sh — Reminder limpio

33 líneas. Bumpea turno vía state_new_turn, imprime reminder de 5 líneas con el checklist y el loop actual, y alerta si hay ≥ 2 strikes acumulados. Limpio, sin side effects más allá del state. El header documenta el bug fixed: antes llamaba state_record_violation (no existía), ahora llama state_record_strike (correcto).

6.9 turnend.sh — Auditoría post-commit

71 líneas. Calcula score, verifica handoff, reporta mínimo (3 líneas), warn si score < 80, reminder handoff si hay loop activo sin handoff. Luego vienen las auditorías post-commit: (1) si el último commit es ≤ 5 min, verifica que PLAN-VIVO § 6 tenga menciones de "L-" (grep -c "L-" ≥ 5 para considerar trazable), (2) si el commit tocó models/migrations/dto, recuerda actualizar STATE-REGISTERS, (3) si tocó frontend/src/.astro/.tsx, recuerda BEHAVIOR-INDEX. Finalmente ajusta closeout_passed según score + handoff.

Bug menor: línea 45 grep -c "L-" "\$JCODE_DIR/iterations/PLAN-VIVO.md" es muy genérico. Cualquier mención de "L-" en el documento cuenta, incluso en texto explicativo. Sería mejor buscar el patrón exacto del ID: grep -cE "^- loop_id: L-[A-Z]+-[0-9]+". Hallazgo **Low** (L-03).

6.10 run_all.sh — Runner con bug crítico

140 líneas. Bien estructurado: declara CATEGORIES como associative array (smoke, diagnostics, audit, measure, validation), soporta --quick y --category, genera evidence JSON con timestamp, guarda logs por test en tests/evidence/<fecha>/<test>.log. El patrón de accumulate total_pass/total_fail/total_skip y construir JSON inline vía python3 -c es portable.

Hallazgo crítico C-01: las categorías referencian 9 tests que **NO EXISTEN** en el filesystem:

```
# En run_all.sh líneas 44-50:
[smoke]="smoke_harness_flow.sh smoke_hook_enforcement.sh smoke_r_aa_1.sh \
    smoke_agent_protocol.sh smoke_r_no_fake_swarm.sh smoke_ddlp_tpsp.sh"
[diagnostics]="diagnostics.sh"
[audit]="audit_harness.sh"
[measure]="measure_harness.sh"

# En el filesystem:
$ ls .jcode/tests/smoke/ .jcode/tests/diagnostics/ .jcode/tests/audit/ .jcode/tests/measure/
# (todas las carpetas están VACÍAS)
```

El runner detecta que no existen (líneas 84-88: "archivo no existe, skip") y reporta SKIP, pero el README afirma que los tests existen y están listos. Esto es engañoso: un usuario que lea el README creerá que el harness está validado, cuando en realidad no tiene **ningún test**. La sección 15.1 propone crear los 9 tests faltantes con esqueletos bash funcionales que validen lo que el README afirma.

Bug menor: línea 35 "--category") CATEGORY="{2:-}"; shift ;; no hace shift del valor (solo del flag). Si el usuario pasa --category smoke, el shift solo consume "--category" y deja "smoke" como siguiente argumento, lo cual no causa bug porque el loop for itera los args restantes, pero es inconsistente con el patrón estándar. Bug **Low** (L-04, agrupado con L-03).

7. Hallazgo crítico — Tests inexistentes

Este hallazgo merece su propia sección porque es el gap más severo del harness. El **README.md** (líneas 90-99) declara explícitamente que el harness provee 6 smoke tests agnósticos que "deben pasar en cualquier proyecto que use el arnés v100-clean":

```
# README.md líneas 90-99:
| Test | Qué verifica |
|-----|-----|
| smoke_harness_flow.sh      | Arnés wirado + vigente |
| smoke_hook_enforcement.sh  | Hooks bash funcionales |
| smoke_r_aa_1.sh            | R-AA-1 anti-autoengaño |
| smoke_agent_protocol.sh    | AGENT-PROTOCOL.md respetado |
| smoke_r_no_fake_swarm.sh   | R-NO-FAKE-SWARM respetado |
| smoke_ddlp_tpsp.sh         | DDLP + TPSP respetados |
```

Sin embargo, al inspeccionar el filesystem:

```
$ find .jcode/tests -type f -name "*.sh"
.jcode/tests/run_all.sh
# (eso es todo — solo el runner, sin NINGÚN test)

$ ls .jcode/tests/smoke/ .jcode/tests/diagnostics/ \
    .jcode/tests/audit/ .jcode/tests/measure/
.jcode/tests/audit/:      (vacío)
.jcode/tests/diagnostics/: (vacío)
.jcode/tests/measure/:    (vacío)
.jcode/tests/smoke/:      (vacío)
```

Impacto: cualquier proyecto que arranque con este template creará tener una batería de tests que valide el arnés, cuando en realidad no la tiene. Si el agente o el usuario corren `bash .jcode/tests/run_all.sh`, verán 6 SKIP en smoke y 1 SKIP en cada una de `diagnostics/audit/measure`, pero el JSON de evidence reportará "status: SKIP" para todas las categorías. El comando `run_all.sh` sale con exit 0 (porque skip no es fail), lo cual es aún más peligroso: un CI que ejecute este runner creará que todo está bien.

Recomendación inmediata: o bien (a) crear los 9 tests con esqueletos bash funcionales (sección 15.1), o bien (b) eliminar las referencias a tests inexistentes del README y de `run_all.sh`, dejando las categorías vacías explícitamente marcadas como "TODO: poblar por el proyecto". La opción (a) es preferible porque el harness gana valor real; la opción (b) es la mínima para no engañar al usuario.

8. Auditoría de agnosticismo

El harness se declara "agnóstico al dominio" — cero paths de proyecto, cero contraseñas, cero reglas de negocio en .jcode/. La auditoría verifica esta declaración escaneando los archivos del arnés en busca de: paths absolutos, nombres de proyecto específicos, stack lock-in (extensiones o rutas hardcodeadas), y referencias a otros runtimes (.claude/, subagent_type=oracle, etc.).

Dimensión	Hallazgo	Cumple	Acción
Paths absolutos	mcp.json línea 11 y 22 usan \${PROJECT_ROOT} y \${HOME} — variables de entorno, no paths hardcodeados. Bien.	Cumple	—
Nombres de proyecto	config.toml [project] name = "proyecto-01" es el placeholder. contamination_patterns.txt está vacío (espera que el usuario lo pueble).	Parcial	El nombre "proyecto-01" es claramente placeholder, aceptable. Pero sería más limpio usar name = "\${PROJECT_NAME:-untitled}".
Stack lock-in (extensiones)	posttool.sh líneas 30 y 104 + turnend.sh líneas 54-58 hardcodean extensiones .go .ts .astro .tsx .jsx .sql .svelte .css .scss .vue. Asume stack web frontend+backend.	No cumple	Hallazgo H-02. Debería estar en config.toml [workspace] code_extensions = [...].
Stack lock-in (paths)	INTERPRETACION.md § 3 usa backend/handlers/ y frontend/src/ en ejemplos de grep. skill worker-ejecutor SKILL.md ídem.	No cumple	Hallazgo Medium M-05. Debería usar placeholders o referenciar [workspace] source_dirs.
Referencias a otros runtimes	No se encontraron referencias a .claude/, subagent_type=oracle, u otros runtimes específicos. Limpio.	Cumple	—
Contraseñas/credenciales	AGENTS.md § 6 declara credenciales dev como placeholders <admin@dev.local> / <dev123>. mcp.json no tiene secrets. Limpio.	Cumple	—
Tablas DB específicas	contamination_patterns.txt incluye instrucciones explícitas de NO agregar patrones genéricos. Buen diseño.	Cumple	—

Tabla 9. Matriz de agnosticismo.

Conclusión: el harness es agnóstico en lo declarativo (PRINCIPLES, AGENT-PROTOCOL, LOOPS, README, AGENTS.md, PROJECT.md, PDR.md son plantillas limpias) pero **no es agnóstico en lo ejecutable:** los hooks y scripts de diagnóstico tienen acoples residuales a un stack web frontend+backend. Para un proyecto Go puro, Python, Rust, o Java, los

detectores R-FAKE-COORDINATOR y la auditoría post-commit de turnend.sh no funcionarán correctamente. Este es el hallazgo más importante después de los tests inexistentes.

9. Análisis del compliance scoring

El sistema de compliance scoring es uno de los activos más valiosos del harness. Convierte comportamientos cualitativos (¿el agente hizo SRSI? ¿escribió handoff? ¿cumplió DDLP?) en un score 0-100 que el agente puede monitorear y optimizar. La sección 6.2 ya cubrió la robustez técnica de state_manager.sh; esta sección analiza el diseño del scoring mismo.

9.1 Pesos y balance

Campo	Peso	Significado	Evaluación
reads_this_turn	15	≥3 lecturas por turno. Incentiva lectura antes de acción.	Razonable. Incentivo correcto.
cumulative_reads	15	Lecturas acumuladas > 0. Incentiva continuidad.	Binario — todo o nada. Podría graduarse por tramos.
drill_done_this_turn	15	El agente "drilló" en el problema (declaración manual).	Subjetivo. No hay verificación.
srsi_done_this_turn	15	grep ejecutado este turno (auto-detectado por hook).	Verificado automáticamente. Buen peso.
ddlp_done	10	Leyó PLAN-VIVO § 3 y § 4 (declaración manual).	Subjetivo. Debería bajar peso o verificarse.
r3s	10	0 strikes = 10, 1-2 = 5, 3+ = 0.	Penalización progresiva correcta.
handoff_written	10	Escribió handoff al cerrar sesión (declaración manual).	Subjetivo. Podría verificarse via git diff.
clean	10	aa1_violations vacío.	Verificado automáticamente. Buen peso.

Tabla 10. Análisis de pesos del compliance score.

9.2 Sesgo actividad vs outcome

El scoring actual tiene un sesgo hacia **actividad** (reads+cumul+drill+srsi = 60 puntos, 60% del total) sobre **outcome** (ddlp+r3s+handoff+clean = 40 puntos, 40%). Esto puede incentivar al agente a "parecer ocupado" (hacer muchas lecturas, declarar drill_done) sin necesariamente producir outcome real (cerrar loops, escribir handoffs honestos, mantener clean). Un agente estratégicamente perezoso podría llegar a score 75 haciendo lecturas superficiales sin tocar código.

Recomendación: rebalancear a 50/50. Por ejemplo: reads=10, cumul=10, drill=10, srsi=15 (srsi es el más verificable, subir peso), ddlp=10, r3s=15 (subir peso, el anti-autoengaño es crítico), handoff=15 (subir, es el outcome más importante), clean=15 (subir, violaciones son indicador directo de mala práctica). Total=100. Hallazgo **Low** (L-02).

10. Análisis del detector de contaminación

El detector de contaminación es el mecanismo que hace cumplir el principio de separación arnés/proyecto. Tiene dos componentes: harness.sh check (auditoría rápida, sin side effects) y clean-contamination.sh (tooling de limpieza manual con 8 subcomandos). Esta sección analiza el diseño del detector y los bugs encontrados.

10.1 Diseño conceptual

El concepto central es excelente: declarar una lista de patrones prohibidos en contamination_patterns.txt (uno por línea, comentarios con #), excluir los paths que legítimamente pueden contener strings del proyecto (tests/, state/, logs/, iterations/, docs/, scratch/), y escanear el resto de .jcode/ con ripgrep. Cuando un patrón aparece en un archivo del arnés, el archivo debe moverse al proyecto o limpiarse. La distinción scan-harness (excluye proyecto) vs scan-project (incluye solo proyecto) es pedagógica y operacionalmente útil.

10.2 EXCLUDE_PATHS — patrón elegante

El array EXCLUDE_PATHS en harness.sh líneas 132-151 lista paths relativos a .jcode/ que no se auditan: tests/**, scratch/**, state/**, logs/**, docs/**, iterations/archive/**, iterations/PLAN-VIVO.md, iterations/PLAN-VIVO.template.md, iterations/INDEX.md, iterations/STATE/**, ARCH/**, REM/**, REV/**, PLAN/**, OP/**, SPRINT-F-LOOP.md, visualizer/server/node_modules/**, visualizer/server/package-lock.json. El razonamiento es correcto: estos paths SON del proyecto (tests, estado runtime, logs, iteraciones) y se espera que contengan strings del dominio. Auditarlos como contaminación sería falso positivo garantizado.

10.3 Bug en clean-contamination.sh

Hallazgo M-04: la función build_exclude_args() en líneas 118-127 retorna los args como string vía echo "\${args[@]}". Esto colapsa elementos con espacios. Cuando se consume en cmd_scan_harness() línea 207 con rg_in_jcode "\$pattern" \$exclude_args (sin comillas en \$exclude_args), los globs como "iterations/PLAN-VIVO.md" se pasan correctamente al ser palabras separadas, pero los paths con espacios (ninguno en la lista actual, pero podría haberlos) se romperían. El patrón correcto en bash es usar **nameref** (declare -n) o una variable global. El código funciona por ahora porque ningún EXCLUDE_PATH tiene espacios, pero es frágil.

```
# Bug en clean-contamination.sh líneas 118-127:
build_exclude_args() {
  local args=()
  for p in "${PROJECT_PATHS[@]}; do
    args+=( --glob "$p" )
  done
  args+=( --glob "!contamination_patterns.txt" )
  args+=( --glob "!clean-contamination.sh" )
  echo "${args[@]}" # ← BUG: colapsa elementos con espacios
```



```
}

# Fix propuesto (nameref):
build_exclude_args() {
    local -n out=$1
    out=()
    for p in "${PROJECT_PATHS[@]}; do
        out+=( --glob "!$p" )
    done
    out+=( --glob "!contamination_patterns.txt" )
    out+=( --glob "!clean-contamination.sh" )
}

# Uso:
local exclude_args=()
build_exclude_args exclude_args
rg_in_jcode "$pattern" "${exclude_args[@]}"
```

11. Análisis de hooks — eficacia de detección

Los 4 hooks son el corazón del enforcement del harness. Sin hooks, PRINCIPLES.md es solo declaración; con hooks, los principios se convierten en mecanismos verificables. Esta sección evalúa la eficacia real de detección de cada hook, identificando falsos positivos, falsos negativos, y edge cases.

Hook	Función	Riesgo	Evaluación
sessionstart.sh	Init state + integrity check + git restore de hooks faltantes.	Bajo. Solo falla si state_manager.sh no puede escribir.	Buen self-repair: si falta un hook, lo restaura de git.
turn_start.sh	Bumpear turno + reminder + alerta strikes ≥ 2 .	Bajo. Solo state_new_turn.	Reminder útil pero no enforcement.
posttool.sh	4 detectores: reads tracking, SRSI, commit-sin-SRSI, R-FAKE-COORDINATOR.	Medio. Bug en regex H-01 + hardcoding extensiones H-02.	El más complejo. Necesita fixes urgentes.
turnend.sh	Score + handoff reminder + auditoría post-commit + closeout.	Bajo. Solo state_se t.	Buen diseño. Bug menor en grep -c "L-".

Tabla 11. Scorecard de hooks.

11.1 posttool.sh — análisis profundo

Es el hook más importante y el que más bugs tiene. Las 5 funciones principales son: (1) log del tool call a logs/post_tool.log, (2) tracking reads via state_record_read, (3) SRSI detection via case sobre tool_name, (4) commit sin SRSI previo $\rightarrow$ strike R-AA-1, (5) swarm spawn tracking + R-FAKE-COORDINATOR detection. El helper is_docs_only_commit() es elegante: detecta commits de documentación para skip los detectores (un commit docs-only no necesita SRSI previo ni swarm).

Edge case no cubierto: el detector R-FAKE-COORDINATOR asume que cualquier commit a código fuente requiere swarm worker previo. Pero ¿qué pasa con commits del coordinator que son legítimamente inline? El bypass clause --coord-self-authorize en línea 100 cubrió este caso, pero depende de que el agente recuerde agregar el flag manualmente. Sería mejor un allowlist de paths donde el coordinator puede committear inline (por ejemplo, . jcode/state/, PLAN-VIVO.md) sin necesidad del flag.

11.2 turnend.sh — auditoría post-commit

La auditoría post-commit (líneas 40-60) es un patrón interesante: usa git_age.sh para detectar si el último commit es reciente (≤ 5 min), y si lo es, verifica que PLAN-VIVO § 6 tenga trazabilidad (grep -c "L-" ≥ 5), y recuerda actualizar STATE-REGISTERS si el commit tocó models/migrations/dto, o BEHAVIOR-INDEX si tocó frontend/src/.astro/.tsx. La intención es buena: atar commits a documentación. Pero la verificación es débil (cualquier "L-" cuenta) y los paths son hardcoded (ver H-02).

12. Matriz de hallazgos priorizada

Consolidación de todos los hallazgos identificados en las secciones anteriores, ordenados por severidad. Cada hallazgo tiene ID estable (no se reasigna aunque se cierre), archivo y línea de referencia cuando aplica, y recomendación específica que se desarrolla en la sección 13.

Sev	ID	Hallazgo	Ubicación	Recomendación
Critic al	C-01	run_all.sh referencia 9 tests inexistentes. El runner reporta SKIP pero README afirma que existen. CI sale exit 0 creyendo que todo pasó.	run_all.sh:44-50 + README.md: 90-99	Crear los 9 tests con esqueletos bash (ver 13.1).
High	H-01	Bug en regex de posttool.sh línea 104. Comilla doble sin escapar rompe el matcher. R-FAKE-COORDINATOR produce falsos negativos.	posttool.sh:104	Usar clase de caracteres [^A-Za-z0-9] (ver 13.2).
High	H-02	Stack lock-in: extensiones .go .ts .astro .tsx .jsx .sql .svelte hardcodeadas en posttool.sh + turnend.sh + INTERPRETACION.md + worker-ejecutor/S KILL.md. Proyectos Python/Rust/Go-puro/Ja va no detectados.	posttool.sh:30,104; turnend.sh: 54-58; INTERPRETACION.md § 3	Extraer a config.toml [workspace] code_extensions (ver 13.3).
Mediu m	M-01	R-3STRIKE-MVP no distingue reglas. Strikes de reglas diferentes se acumulan al mismo contador. El principio original habla de "mismo fallo, mismo método".	state_manager.sh:165-179	Agrupar strikes por regla en aal_violations[].rule.
Mediu m	M-02	R-DOS-PLANOS sin enforcement. Ningún script mide longitud de PLAN-VIVO.md. La regla de archivar a >350 líneas es puramente convencional.	PRINCIPLES.md § 2	Agregar check en turnend.sh: wc -l PLAN-VIVO.md.
Mediu m	M-03	DDLP sin verificación. El agente puede marcar ddlp_done=true sin leer PLAN-VIVO. No hay enforcement real.	state_manager.sh + posttool.sh	Hook que detecte tool de lectura sobre PLAN-VIVO.md.
Mediu m	M-04	build_exclude_args retorna string en clean-contamination.sh. Funciona por ahora pero es frágil.	clean-contamination.sh:118-127	Usar nameref (declare -n) o array global.
Mediu m	M-05	Paths backend/handlers/ y frontend/src/ hardcodeados en INTERPRETACION.md § 3 y worker-ejecutor/SKILL.md como ejemplos de grep.	INTERPRETACION.md:53-58; worker-ejecutor/SKILL.md:23-26	Reemplazar con placeholders <source_dir>.
Low	L-01	Detección "SRSI en últimos 5 min" usa tail -50 en vez de parsear timestamps ISO 8601 del log.	posttool.sh:66-67	Parsear timestamps con awk/date.
Low	L-02	Sesgo actividad vs outcome en scoring: 60/40. Podría incentivar "parecer ocupado" sin outcome real.	state_manager.sh:230-282	Rebalancear a 50/50 (ver 9.2).

Sev	ID	Hallazgo	Ubicación	Recomendación
Low	L-03	grep -c "L-" muy genérico en turnend.sh. Cualquier mención de "L-" cuenta como trazabilidad.	turnend.sh:45	Usar grep -cE "^-loop_id: L-[A-Z]+-[0-9]+"
Low	L-04	--category no shift del valor en run_all.sh. Bug menor, no causa problema runtime.	run_all.sh:35	Agregar shift 2 en el case branch.

Tabla 12. Matriz completa de hallazgos.

13. Recomendaciones accionables con código

Para los hallazgos más críticos se proponen fixes concretos con código listo para aplicar. Los snippets están pensados como patches que reemplazan las secciones existentes sin requerir refactor mayor.

13.1 Fix C-01 — Crear los 9 tests faltantes

Crear 9 scripts bash funcionales que validen lo que el README afirma. Cada test debe ser: (1) standalone (no depende de otros), (2) exit 0 en pass / exit 1 en fail, (3) output legible (pass/fail/info), (4) side-effect free salvo en tests que validen side effects. Esqueleto para smoke_harness_flow.sh:

```
#!/usr/bin/env bash
# .jcode/tests/smoke/smoke_harness_flow.sh
# Valida: arnés wirado + vigente
set -uo pipefail
REPO_ROOT="${REPO_ROOT:-$(git rev-parse --show-toplevel 2>/dev/null || pwd)}"
JCODE_DIR="$REPO_ROOT/.jcode"

pass() { printf " PASS %s\n" "$*"; }
fail() { printf " FAIL %s\n" "$*" >&2; exit 1; }

# 1. Estructura canónica
for f in PRINCIPLES.md AGENT-PROTOCOL.md LOOPS.md README.md \
    config.toml mcp.json; do
    [[ -f "$JCODE_DIR/$f" ]] || fail "falta $f"
done
pass "archivos canónicos presentes"

# 2. Hooks ejecutables
for h in sessionstart turn_start turnend posttool; do
    [[ -x "$JCODE_DIR/hooks/$h.sh" ]] || fail "hook $h.sh no ejecutable"
done
pass "4 hooks ejecutables"

# 3. Compliance.json válido
python3 -c "import json;json.load(open('$JCODE_DIR/state/compliance.json'))" \
    || fail "compliance.json inválido"
pass "compliance.json válido"

# 4. Score sumando 100
score=$(bash "$JCODE_DIR/lib/state_manager.sh" score 2>/dev/null || echo 0)
[[ "$score" =~ ^[0-9]+$ ]] || fail "score no es numérico: $score"
[[ "$score" -le 100 ]] || fail "score > 100: $score"
pass "score $score/100 en rango"

echo "smoke_harness_flow: OK"
exit 0
```

Los otros 8 tests siguen el mismo patrón, validando: smoke_hook_enforcement.sh (que los hooks detectan SRSI missing, swarm spawns, R-FAKE-COORDINATOR); smoke_r_aa_1.sh (que un commit sin grep previo produce strike); smoke_agent_protocol.sh (que los 11 items del checklist tienen su mecanismo); smoke_r_no_fake_swarm.sh (que commit a código sin

swarm reciente produce strike); smoke_ddlp_tpsp.sh (que la estructura § 3/ § 4/ § 5 dePLAN-VIVO existe); diagnostics.sh (corre harness.sh check + breakdown); audit_harness.sh(valida 0 contaminación en arnés puro); measure_harness.sh (mide tiempos deinit/check/score).

13.2 Fix H-01 — Regex de posttool.sh

El bug está en la línea 104 de posttool.sh. La regex original:

```
# BUG: comilla doble sin escapar dentro de single-quote
if echo "$tool_args" | grep -qE '\.(go|ts|astro|tsx|jsx|sql|svelte|css|scss|vue)( |"$|$)'; then
```

Fix propuesto usando una clase de caracteres explícita en lugar de mezclar comillas:

```
# FIX: usar clase de caracteres [^A-Za-z0-9] para "espacio o fin"
# y construir el patrón en una variable para legibilidad
CODE_EXT_PATTERN='\.(go|ts|astro|tsx|jsx|sql|svelte|css|scss|vue)([^A-Za-z0-9]|$)'
if echo "$tool_args" | grep -qE "$CODE_EXT_PATTERN"; then
```

Aplicar el mismo fix a la línea 30 en is_docs_only_commit() que tiene el mismo patrón. Idealmente, extraer la lista de extensiones a una variable de configuración (ver 13.3).

13.3 Fix H-02 — Stack-agnóstico vía config

Mover las extensiones hardcodeadas a config.toml para que cada proyecto declare su stack. Agregar sección a config.toml:

```
# .jcode/config.toml — agregar sección
[workspace]
source_dirs = ["src", "docs", "tests"]
exclude_dirs = ["node_modules", ".git", "dist", "build"]
# Extensiones de código fuente para detectores R-FAKE-COORDINATOR
# y auditoría post-commit. Ajustar al stack del proyecto.
code_extensions = [".go", ".ts", ".astro", ".tsx", ".jsx",
                  ".sql", ".svelte", ".css", ".scss", ".vue"]
# Para un proyecto Python: [".py", ".ipynb", ".sql"]
# Para Rust: [".rs", ".toml"]
# Para Go puro sin frontend: [".go", ".sql"]
```

Helper bash para leer la lista (sin depender de jq ni de un parser TOML completo — usamos sed + tr que es universal):

```
# .jcode/lib/config_reader.sh — nuevo helper
#!/usr/bin/env bash
# Lee arrays desde config.toml sin parser TOML completo.
set -uo pipefail

CONFIG_FILE="${JCODE_CONFIG:-.jcode/config.toml}"

# Uso: config_get_array "workspace.code_extensions"
# Retorna: palabras separadas por espacio, listas para iterar
config_get_array() {
    local key="$1"
    local section="${key%.*}"
    local field="${key#*.*}"
    # Buscar la línea "field = [ ... ]" dentro de la sección [section]
```

```

awk -v sec="[$section]" -v fld="$field" \
  '$0 == sec { in_sec=1; next } /\[/ { in_sec=0 } in_sec && $1 == fld "=" { sub(/\^[^=]*=s*\[/, ""); sub(/
\].*$/, ""); gsub(/["'\,\s]+/, " "); print; exit }' \
  "$CONFIG_FILE" | tr ' ' '\n' | grep -v '^$'
}

# Uso en posttool.sh:
# source .jcode/lib/config_reader.sh
# EXTENSIONS=$(config_get_array workspace.code_extensions)
# PATTERN=$(echo "$EXTENSIONS" | sed 's/\./\\./' | paste -sd'|' -)
# if echo "$tool_args" | grep -qE "($PATTERN)([A-Za-z0-9]|$)"; then ...

```

13.4 Fix M-01 — Strikes agrupados por regla

Modificar `state_record_strike()` para contar strikes por regla individualmente, y activar R-3STRIKE-MVP solo cuando la misma regla acumula 3 strikes. Esto respeta el principio literal "mismo fallo, mismo método":

```

# state_manager.sh — nueva función
state_strikes_for_rule() {
  local rule="$1"
  python3 -c "
import json
d = json.load(open('$STATE_FILE'))
viol = d.get('aa1-violations', [])
print(sum(1 for v in viol if isinstance(v, dict) and v.get('rule') == '$rule'))
"
}

# En state_record_strike, antes de activar R-3STRIKE-MVP:
local rule_count=$(state_strikes_for_rule "$rule")
if [[ "$rule_count" -ge 3 ]]; then
  echo "[state] R-3STRIKE-MVP triggered for rule=$rule (3 strikes del mismo tipo)"
  state_set r_3strike_mvp_activations "$(( $(state_get r_3strike_mvp_activations) + 1 ))"
fi

```

13.5 Fix M-02 — Enforcement de R-DOS-PLANOS

Agregar a `turnend.sh` un check de longitud de `PLAN-VIVO.md`. Si supera 300 líneas, advertir; si supera 400, bloquear `closeout`:

```

# turnend.sh — agregar después de línea 60
PLAN_VIVO="$JCODE_DIR/iterations/PLAN-VIVO.md"
if [[ -f "$PLAN_VIVO" ]]; then
  lines=$(wc -l < "$PLAN_VIVO")
  if [[ "$lines" -gt 400 ]]; then
    echo "[turnend] ⚠ PLAN-VIVO.md tiene $lines líneas (>400). BLOQUEAR closeout — archivar secciones viejas a iterations/archive/PLAN-VIVO-$(date +%Y-%m-%d).md"
    state_set closeout_passed false
    return 1
  elif [[ "$lines" -gt 300 ]]; then
    echo "[turnend] ⚠ PLAN-VIVO.md tiene $lines líneas (>300). Considerar archivar pronto."
  fi
fi

```

14. Veredicto final y roadmap

14.1 Score global

El harness jcode v100-clean obtiene un score global de **7.5 / 10**. La filosofía es excelente (separación estricta arnés/proyecto, compliance scoring, anti-autoengaño codificado, discipline documental clara), pero la ejecución tiene dos gaps críticos (tests inexistentes, bug en regex de posttool.sh) y varios gaps menores (stack lock-in residual, DDLP sin verificación, scoring con sesgo actividad). Para ser production-ready necesita un sprint de hardening de aproximadamente 2-3 días de trabajo enfocado.

Dimensión	Score	Comentario
Filosofía y diseño	9 / 10	Principios claros, separación arnés/proyecto, compliance scoring, anti-autoengaño.
Calidad de scripts bash	8 / 10	clean-contamination.sh y state_manager.sh son modelo. posttool.sh tiene bugs.
Cumplimiento PRINCIPIOS PARTE I	7 / 10	SRSI y R-NO-FAKE-SWARM bien. R-DOS-PLANOS, DDLP, TPSP solo declarativos.
Cumplimiento PRINCIPIOS PARTE II	8 / 10	Task classification y loop format completos. Stop conditions sin enforcement.
Agnosticismo real	6 / 10	Limpio en lo declarativo. Stack lock-in residual en hooks y skills.
Testing y validación	2 / 10	Runner existe pero 9 tests inexistentes. Sin CI. Sin tests de regresión.
Observabilidad	5 / 10	Compliance score + visualizer. Sin métricas de runtime, sin alertas.
Documentación	9 / 10	README, PRINCIPIOS, AGENT-PROTOCOL, LOOPS, INDEX muy completos. Bug fixes documentados en headers.

Tabla 13. Scorecard por dimensión.

14.2 Roadmap priorizado

Prioridad	Esfuerzo	Tarea	Valor
P0	1-2 días	Crear los 9 tests faltantes (C-01). Sin esto el harness miente al usuario. Aplicar esqueleto de 13.1.	Crítico
P0	30 min	Fix bug regex posttool.sh (H-01). Aplicar patch de 13.2.	Crítico
P0	2 horas	Extraer code_extensions a config.toml (H-02). Aplicar helper de 13.3 + actualizar posttool.sh + turnend.sh.	Crítico
P1	4 horas	Crear GitHub Actions workflow que corra run_all.sh en PR. Agregar badge a README. Falta CI integration mencionado en sección 6.	Alto

Prioridad	Esfuerzo	Tarea	Valor
P1	2 horas	Fix M-04 en clean-contamination.sh (build_exclude_args via nameref). Aplicar patch de 13.3.	Alto
P1	3 horas	Fix M-01, M-02, M-03. Strikes por regla, enforcement R-DOS-PLANOS, verificación DDLP. Aplicar 13.4 y 13.5.	Alto
P2	1 día	Tests de regresión para state_manager scoring. Tests que verifiquen que score=100 con todo en true, score=0 con todo en false, score=50 con la mitad.	Medio
P2	1 día	Observabilidad extendida: exportar compliance.json a Prometheus/Grafana o similar. Métricas de runtime por loop.	Medio
P2	2 días	Visualizer integration: verificar que el servidor SSE/WS del visualizer consume bien los eventos. Falta validación.	Medio
P3	1 día	Rebalancear pesos de scoring a 50/50 actividad/outcome (L-02).	Bajo
P3	2 horas	L-01, L-03, L-04. Detección de SRSI por timestamp, grep -cE más específico, shift 2 en --category.	Bajo

Tabla 14. Roadmap priorizado de hardening.

14.3 Conclusión

El harness jcode v100-clean es un excelente punto de partida para cualquier proyecto nuevo bajo jcode. La filosofía de separación arnés/proyecto, el compliance scoring 0-100, y la disciplina documental (AGENTS.md + PROJECT.md + PDR.md + .jcode/ aislado) son patrones que vale la pena adoptar. Sin embargo, no es production-ready tal cual está: los 9 tests inexistentes (C-01) y el bug en la regex de posttool.sh (H-01) son gaps que deben cerrarse antes de usar el template en un proyecto real. Con un sprint de 2-3 días aplicando los fixes P0 y P1 del roadmap, el harness pasa de 7.5/10 a 9.0/10 y se convierte en un activo confiable.

La acotación del usuario sobre "usar comandos o scripts por encima de la lógica de los agentes" se refleja plenamente en este análisis: los scripts bien diseñados (clean-contamination.sh, state_manager.sh, el dispatcher) son los puntos fuertes del harness, mientras que las reglas puramente declarativas (DDLP, TPSP, R-DOS-PLANOS) son los puntos débiles precisamente por depender de la honestidad del agente. La recomendación general es **convertir toda convención declarativa en un script que la verifique**: si una regla no tiene un hook o script que la haga cumplir, no existe operativamente.